FASE 2 · PROYECTO 05 · PROYECTO ESTRELLA

ringdev

Un character device driver:
buffer circular en el kernel, bloqueante, concurrente y con poll

Versión 1.0 · Roadmap Sistemas Linux

Resumen

Vas a escribir un driver de dispositivo de caracteres que expone `/dev/ringdev`: un buffer circular dentro del kernel en el que varios procesos escriben y leen a la vez. Tendrás que copiar datos de forma segura entre user space y kernel space, dormir y despertar procesos, sincronizar accesos concurrentes y soportar `poll`, `ioctl` y `O_NONBLOCK`. Es el proyecto que demuestra que puedes trabajar en Ring 0.

CAPÍTULO I

Introducción

En Linux, un *character device* es un fichero especial (`/dev/algo`) identificado por un número *major* (qué driver lo gestiona) y un *minor* (qué instancia). Cuando un proceso hace `read` sobre él, la VFS acaba llamando a la función `read` de la `struct file_operations` que tu driver registró. Tu código corre en el contexto del proceso que hizo la llamada, en Ring 0, con un puntero a memoria de user space del que **no puedes fiarte**.

`ringdev` se comporta como un pipe con nombre implementado por ti: los escritores bloquean si el buffer está lleno, los lectores bloquean si está vacío, y todo debe seguir siendo correcto con 16 procesos golpeándolo a la vez desde varias CPU.

CAPÍTULO II

Instrucciones generales

- Mismas reglas que `ring0`: VM obligatoria, estilo del kernel, `checkpatch.pl` limpio, cero warnings.
- Muy recomendable: trabaja con el kernel de depuración del bonus de `ring0` (KASAN + lockdep + kmemleak). En la evaluación se cargará tu módulo en un kernel así, y cualquier informe de esas herramientas es un fallo.
- Entrega: `driver/` (módulo), `include/ringdev.h` (cabecera compartida con user space), `tools/` (utilidades de user space) y `tests/` (scripts de prueba).
- Toda ruta de error en la inicialización debe deshacer exactamente lo que se hizo, en orden inverso (el patrón de `goto` de limpieza del kernel).

CAPÍTULO III

Parte obligatoria

1. Registro del dispositivo

- Obtén un major dinámico con `alloc_chrdev_region`, registra el `cdev` y crea la clase y el nodo con `class_create` y `device_create` para que udev cree `/dev/ringdev` automáticamente.
- Atención: la firma de `class_create` cambió en el kernel 6.4. Consulta la de tu versión en elixir.bootlin.com.
- Añade una regla de udev en `tools/99-ringdev.rules` para que el nodo tenga permisos 0666.
- Pon `.owner = THIS_MODULE` en tus `file_operations` y comprueba qué ocurre con `rmmod` mientras un proceso tiene el dispositivo abierto.

2. Semántica de lectura y escritura

Un único buffer circular compartido por todos los que abren el dispositivo, de tamaño `bufsize` (parámetro del módulo, por defecto 4096 bytes, entre 64 y 1 MiB).

Situación	Comportamiento
`read` con datos disponibles	Copia `min(pedido, disponibles)` bytes y devuelve cuántos. Nunca bloquea si hay al menos un byte.
`read` con el buffer vacío	Bloquea hasta que haya datos. Con `O_NONBLOCK`: `-EAGAIN`.
`write` con espacio disponible	Copia todo lo que quepa (al menos 1 byte) y devuelve cuántos: escrituras parciales, como en `rawio`.
`write` de ≤ `RINGDEV_ATOMIC` (256) bytes	**Atómica**: se escribe entera o se bloquea hasta que quepa entera. Nunca se intercala con otro escritor.
`write` con el buffer lleno	Bloquea. Con `O_NONBLOCK`: `-EAGAIN`.
Señal durante un bloqueo	La espera se interrumpe y la llamada devuelve `-ERESTARTSYS` (o los bytes ya transferidos).
Puntero de user space inválido	`-EFAULT`. Nunca un oops.
`lseek`	No permitido: el dispositivo no es *seekable* (`nonseekable_open`).

> **NOTA**
> La garantía de atomicidad para escrituras pequeñas existe también en los pipes (`PIPE_BUF`, `man 7 pipe`). Léelo: estás implementando la misma idea y sus motivos son los mismos.

3. Concurrencia

- Protege el estado del buffer con un `mutex`. Justifica en el README por qué un `spinlock` no sirve aquí (pista: ¿puede dormir `copy_from_user`?).
- Usa dos *wait queues* (lectores y escritores) con `wait_event_interruptible`. Comprueba siempre la condición **con el lock tomado** tras despertar: otro proceso puede haberte adelantado.
- Nunca duermas con el mutex tomado.
- Al copiar a/desde un buffer circular, la región puede estar partida en dos tramos (fin y principio del array). Manéjalo sin copias intermedias innecesarias.

4. poll e ioctl

- Implementa `.poll`: `EPOLLIN | EPOLLRDNORM` si hay datos, `EPOLLOUT | EPOLLWRNORM` si hay espacio. Debe funcionar con `poll`, `select` y `epoll` desde user space.
- En `include/ringdev.h`, define los ioctl con las macros `_IO`, `_IOR` y `_IOW` y un número mágico que no colisione (consulta `Documentation/userspace-api/ioctl/ioctl-number.rst`):

ioctl	Efecto
`RINGDEV_GET_SIZE`	Devuelve la capacidad del buffer.
`RINGDEV_GET_USED`	Devuelve los bytes ocupados.
`RINGDEV_CLEAR`	Vacía el buffer y despierta a los escritores.
`RINGDEV_RESIZE`	Cambia la capacidad conservando los datos. Si no caben, `-EBUSY`. Requiere `CAP_SYS_ADMIN`.
Cualquier otro	`-ENOTTY`.

> **PISTA**
> Para `RESIZE`, reserva la memoria nueva **antes** de tomar el lock (puede dormir y puede fallar), copia y cambia los punteros con el lock tomado, y libera la memoria antigua después.

5. Herramientas de user space

- `tools/rdctl`: interfaz de línea de comandos para los ioctl (`rdctl size`, `rdctl used`, `rdctl clear`, `rdctl resize N`).
- `tools/rdstress`: lanza W hilos escritores y R hilos lectores durante S segundos. Cada escritor envía registros de 16 bytes `{writer_id, secuencia, checksum}`. Los lectores validan cada registro: checksum correcto y, para cada escritor, secuencias estrictamente crecientes. Al final se comprueba que no se perdió ni duplicó ningún registro.
- `tools/rdpoll`: un único proceso que, con `epoll`, lee de `/dev/ringdev` y de stdin a la vez.

CAPÍTULO IV

Parte bonus

- Parámetro `ndevs`: crea N dispositivos independientes (`/dev/ringdev0` … `/dev/ringdevN-1`) con minors distintos y estado separado (usa `container_of` en `open` para encontrar el tuyo).
- Atributos de sysfs de solo lectura (`used`, `size`, `total_bytes_written`) con `DEVICE_ATTR_RO`.
- Notificación asíncrona con `.fasync` (señal `SIGIO` al haber datos).
- Estadísticas por CPU con contadores `percpu` y una comparación de rendimiento frente a un contador atómico global.
- Un documento `DESIGN.md` con un diagrama del buffer y un análisis de las race conditions que encontraste.

CAPÍTULO V

Evaluación

Prueba	Esperado
`echo hola > /dev/ringdev; cat /dev/ringdev`	`hola`; luego `cat` bloquea. Ctrl+C lo interrumpe al instante.
`cat /dev/ringdev` en un terminal y `echo` en otro	El lector despierta y muestra los datos.
`dd if=/dev/zero of=/dev/ringdev bs=1M count=1`	Bloquea al llenarse; un lector lo desbloquea.
Lectura y escritura con `O_NONBLOCK`	`EAGAIN` en los casos correctos.
`rdstress -w 8 -r 8 -s 60` en 4 vCPU	0 registros corruptos, perdidos o duplicados.
`rdstress` + `rdctl resize` en bucle	Sin corrupciones ni deadlocks.
Programa que pasa a `read` un puntero inválido o del kernel	`EFAULT`; el sistema sigue intacto.
`rmmod` con el dispositivo abierto	Rechazado (módulo en uso).
Carga con `bufsize=10`	Rechazada con `-EINVAL`.
Kernel con KASAN, lockdep y kmemleak tras todas las pruebas	Ningún informe.
Preguntas: ¿qué hace `copy_to_user` si la página no está mapeada? ¿Qué es `-ERESTARTSYS` y quién lo traduce?	Respuesta precisa del evaluado.